Generative AI use manual — Mark Kato

At the bottom, under “Appendix”, there is further explanation of a few points I felt may be important.

Tool register

Visible messages: **509**
Tool events: **1903**

I mainly used OpenAI Codex, the first time when I began on the 5th of October in our last class at school (rain close-down). Mainly means 95%. The above 2 benchmarks are from 1 of 3 conversations — the main one. I access Codex through T3 Code to save RAM, but this time I used VS Code for the Jupyter file — instead of Sublime Text. I used Codex in every stage, at least to check my work and receive feedback. (Windows proves to be still difficult as the base OS consumes significant RAM, just to turn on the computer.)

Workflow map

After understanding the foundation of the case, I began writing a plan.

I followed my plan step by step, starting with the ipynb file. The majority of the code I got from the Machine Learning course with Germán Sanchez. I told Codex to scan the folder and re-use the code. The majority of which I didn't understand as additions, I looked up online as well as asked Codex to explain. This led to removal of quite a bit, as Codex often "perfects useless *things*".

I do not understand all code — the majority I do — but there was no task completely done by AI. Copying code from the previous folder, which I know was AI, was checked cell by cell. Writing the plan was completely me, and Codex reviewed it — suggested different ordering I implemented. The current manual is "hand-written" as well.

A basic rule I follow in all I do is first action, and over time perfecting the re-occurring problems, because those are the important ones — signalled by their re-occurrence. As such, I started with a large and extensive task for Codex before I touched the case myself, to have a baseline starting point I improve.

Five pivotal interactions

First

The most important interaction is the way Codex is trained. I am working to create an LLM system I can access even if they became much more restricted - combined with the LLM getting to know me and fit my preferences.

Currently, this shows up as RAG training on my computer, with Codex loading into memory natural language files, some of which are;

Every legacy memory path written further down this file is now a FROZEN BACKUP. Do not read them (they are stale) and never write to them. Corrected canonical locations: - Extended profile: `user_extended_profile.md` - Personal-life doc: `arti_life_system.md` - Personal OS: `personal_os_architecture.md` - AI-build corpus: `reference_ai_build_corpus.md` - Global Claude identity file (kept in sync with this one): `CLAUDE.md`

In essence, they give context for the LLM. I have further maps loaded with .codex skills, and rules carefully created. "Never read `.env` files or print secret values (only derived facts/fingerprints)" is an example.

Second

Perhaps the most important in any task is to plan well. Here is my interaction with Codex regarding the plan. I wrote 10 points of what is included in plan.md file, and asked Codex what they think, their response;

Mostly yes. Your logic is good, but there are three important corrections:

1. Split before any transformation that learns statistics.
2. Do not automatically remove outliers.
3. Add a validation/CV stage before the final test.

Corrected workflow

I would have removed outliers right away - a habit I developed, but I realised Codex may be right - that age 78 with a credit line may be rare but nonetheless possible. I implemented their first 2 recommendations.

Third

My first interaction with Codex was understanding the problem, getting context from previous courses, and creating a first draft we can improve right away. Transcript;

User — 2026-09-21 10:04:40 Romance Daylight Time

Check out the "AI" conversation lo que hemos tenido sobre del clase en Esade contigo,

Hay un model que se llama v1_aggregate_model.ipynb > que lo he hecho para ultimo clase el ano pasado. También, hay links a datasets en este archivo,

Find the files, understand them, and help me understnad. I need to verificar si son buenos o no.

Following that, I went ahead to create the first drafts;

User

0. Understand and prove former through stating
1. Train Test Split > 80-20 convention (though I prefer 70-30) > ought to come before scaling > important because without that the model would learn from statistics of the testing set as well > "data leakage”s

(Only on training data 1-6);

1. Cleaning of data > duplicates and null values → understand the dataset, every column. > we remove duplicates because weight of the mean would increase artificially → yes, the mean, mode, variance, st. deviation
2. Remove outliers (IQR or Z-score) - that are actual error! check before, LLM; "In the credit case, a very large loan may be a real and important risk signal. Removing it could make the model less realistic."
3. Check for contradictions and inefficciencies & transform categorical variables to numerical ones with encoding (One-hot or ordinal)
4. Visualise starting data
5. scale / normalize
6. fit() and train for each model > "feed" the models to the dataset
7. predict() onto test dataset and predict for each model > run the models
8. evaluate each model
9. Iterate

Now according to these steps

> Now do draft 3 > Launch 3 sub-agents with 0 context > and as usual, have them report back to you and you only report to me once you iterated over the errors they have identified. Make sure to use the code of those previous notebooks > they work.
>
> As one can imagine, this came out with lots of errors. AI works best when given explicit instructions (the AI we have available). I went through code-block by code-block for verification, and the general rule I follow: understand or don't include.

Fourth

Prompt: “Ok here all fits are done? These? - with image and referring to the fit-transform stages - "Are these all the predictions we made? This confusion matrix? I’d like you to display a confusion matrix.”

Comparing candidate models - I asked about the majority baseline, Logistic Regression, Decision Tree and Random Forest. The majority baseline is a useful piece of code I learned from Codex, which acts as a minimum benchmark - if we don't hit that our models are useless no matter how good the metrics may look.

The response also explained `predict()` and `predict_proba()` then displayed the matrix for the Random Forest, I needed to be sure it is very visual, not just a text-based 2x2 block.

I accepted Random Forest because it produced the highest net value at the fixed 20% human review capacity.

I verified this using the frozen-test results, the confusion matrix from the counts.

Fifth

I've done quite a few runs of "perfecting over time";

Prompt **User — 2026-10-18 15:25:27 Romance Daylight Time**

Make it like this

2. Why is context still right above the code? I want code then writing. code then writing. According to the plan.md
3. This assert frame code what is it?
4. I want the english database first > a great half-the-screen-filling sample then the original
5. Did you take all main code from the previous machine learning courses?
6. You didn't include IQR nor z-score for outlier detection??? Include.
7. Refix the ordering of my plan please
8. Always use the english names when displaying
9. Fix the error in the predict section Go.

Attached image "image.png" is saved at: C:\Users\katom.t3\userdata\attachments\a4f845ea-69b0-4865-8222-8ce310787679-5c7f3807-8ab7-496a-b80c-ffbdb9e782a9.png … 7 more image paths.

Codex often deviated from my original plan.md, so I always wrote that it treats it as canon. After execution I often needed to go back 2–3 times to fix the original problem — which often times was just removing unnecessary code. Verification always ended with re-running the whole ipynb file and seeing the results — whether they make sense or not.

Two Failure Cases

1. Recurring problem: code or text I write gets overwritten by Codex

You keep re-writing the text I delete.

[Attached image "image.png" is saved at: C:\Users\katom.t3\userdata\attachments\a4f845ea-69b0-4865-8222-8ce310787679-55a982da-7b4a-421f-98fe-14a1f2b5234d.png]

This is easy to detect if we check the work of Codex after each prompt. The most annoying above all, when we do the work just for our dear LLM to erase it. All these required manual correction, despite a law being written into RAG (~make sure to always keep text the user writes .... examples;...)

2. Advanced or complicated code used by the LLM

I asked Codex again and again to use code from our previous Machine Learning course — as I was familiar with the contents. Another recurring problem was that I needed to manually copy-paste such code because, quite simply, it took longer for the LLM to paste the right code I was referring to — or it always added something unwanted.

Again, detection came with checking after each response of Codex.

Understanding Check

Our chosen threshold is 20% of candidates with the highest risk-default value. This is preferable because we have a capacity of 1,800 reviews per month. Scaled to 8,000 reviews per month, that is 20% — 20% of reviews can be back-checked by a human. As such, after the model predicts values between 0–1, with 1 being 100% defaults, we order them descending and send the top 20% of rows — the ones most likely to default.

This is preferable to setting a threshold of “if default-risk ratio >= 0.5 we mark for human review” because often that may produce a lot more than 1,800 candidates from 8,000 rows, and gives no more weight to the higher risk than lower risk selections.

Authorship Statement

The dictionary/translation code I did not originally understand was made substantially by Codex. The formatting took some time, but the dataset got translated into English and I verified with Google Translate whether the translations are valid.

There are other parts of the code — specific syntax — which I could not explain in 1 minute, but the overall architecture, the steps and functions I understand. I mostly drew on the knowledge learned in the Machine Learning course with Germán Sanchez.

Appendix

Another prompt that is loaded into context for Codex - as mentioned I am building an LLM system;

Mark is building **Sky** (his partner-AI; engine at `path`. When he says **"feed Sky"** — and proactively before any context wipe — write a rich distillate `.yaml` into `path`, following that folder's `_SCHEMA.yaml`. The `summary:` must be a RICH narrative (judgment + context reached, not bullets). Filename `<YYYY-MM-DD>_<short-kebab-topic>.yaml`. Codex raw-session capture is configured globally in `~/.codex/hooks.json` for `PreCompact` and `SessionEnd`; it copies the available transcript into `sky_memory/imports/sessions/raw/`. The hook is a safety net, not a replacement for the rich manual distillate.